Student Guide — Lab 5: pavo Walkthrough

Quantifying reef-fish color patterns · EEB 187 Week 6

EEB 187 — Ecology & Evolution of Color

May 6, 2026

Table of contents

How to use this guide

This is the **step-by-step companion** to the live demo. Use it when:

- You **fell behind** in lab and need to catch up
- You want to **try the analysis at home** before lab on a different fish
- You're **doing Part B** (your three lineage fish) and want a written reference for each step

It pairs with three other files in your lab repo:

File	What it's for
`scripts/walkthrough.R`	The R script you'll actually run line-by-line
`scripts/helpers.R`	Helper functions (segment_fish, simple_coldists, etc.) — you load these once at session start
`lab-05-pavo-intro.qmd`	The **worksheet** where you fill in your answers and submit

Read this guide top-to-bottom the first time. Once you've done it once, you can use it as a quick lookup.

> Lost? Stuck?
>
> Every step here ends with a **"What to do if it didn't work"** box. Read those before posting in the channel — most fixes are one-line.

---

1. Orient yourself in RStudio

When RStudio opens (whether locally, via Docker, Binder, or Codespaces), it has four panes:

```
TOP-LEFT: Source pane            TOP-RIGHT: Environment
(Where walkthrough.R will        (Variables you've created;
 open after you click it)         empty when you start)
```

```
BOTTOM-LEFT: Console              BOTTOM-RIGHT: Files /
(Where you type R commands        Plots / Help / Packages
 and see the output)              (Click "Files" to navigate)
```

Your starting working directory depends on how you launched RStudio:

- **Docker**: opens in `~/lab` automatically (the lab repo is mounted there)
- **Native**: opens wherever you last left it. Click **Session → Set Working Directory → Choose Directory**, navigate to `eeb187-pavo-lab`, click *Open*.
- **Binder / Codespaces**: opens in the lab repo by default

Confirm where R thinks you are — in the Console (bottom-left), type:

```
getwd()
```

You should see `/home/rstudio/lab` (Docker / Binder / Codespaces) or `/Users/yourname/eeb187-pavo-lab` (Native). If it's something else, you need to fix it before continuing — see the box below.

> ⚠ If `getwd()` doesn't show the lab repo
>
> - **Docker**: stop the container and restart from inside the cloned repo. Run `cd ~/eeb187-pavo-lab` *before* running `docker run`.
> - **Native**: in RStudio, **Session → Set Working Directory → To Project Directory** if you opened the `.Rproj` file. Otherwise navigate manually as above.
> - **Binder**: this should always work — refresh the browser tab.

Find the walkthrough script

Open the **Files pane** (bottom-right). You should see (among others):

- `scripts/` — folder. Click to expand.
  - `walkthrough.R` ← **this is the live-coded script**
  - `helpers.R` — the helper functions
  - `verify_pipeline.R` — runs the full pipeline as a smoke test
- `images/` — folder. Has subfolders `demo/` (the 3 demo fish, pre-loaded) and `student-fish/` (your assigned lineage's 5 starter species).
- `lab-05-pavo-intro.qmd` — the worksheet you'll submit
- `README.md`, `prep.html`, etc. — supporting docs

Click `walkthrough.R` in the Files pane. It opens in the Source pane (top-left).

---

2. Get your images in place

The lab uses two sets of images: the three **demo fish** (already loaded) and **your three lineage fish** (you'll either use the pre-loaded library or upload your own).

Demo fish — already loaded, nothing to do

Three demo fish — sergeant major, cleaner wrasse, mandarin fish — live at `images/demo/segmented/`. They're pre-segmented (background removed, white-padded, standardized canvas). You'll use these in **Part A** of the worksheet.

Your lineage's fish — pick one of two paths

Path A — Use the pre-loaded library (zero uploading, recommended)

The lab repo already includes **5 starter species per lineage** in `images/student-fish/<lineage>/`. If three of those span the pattern types you want (vertical bars, horizontal stripe, reticulate/spotty), you don't need to upload anything.

In the R Console, list what's in your lineage:

```
list.files("images/student-fish/chaetodontidae")    # change lineage as needed
```

Pick three, write their filenames in your worksheet, and skip Path B.

Path B — Bring your own fish photos

You'll do this in two steps: (1) name your photo correctly, (2) get it into the `images/student-images/` folder. The "how" is different per launch path.

> **i** Why a separate `student-images/` folder?
>
> The lab repo has three image folders:
>
> - `images/demo/` — the 3 demo fish for Part A (don't touch)
> - `images/student-fish/<lineage>/` — the **pre-loaded starter species** (don't add your own files here, so you can always tell what's curated vs. what's yours)
> - `images/student-images/` — **this is where YOUR uploads go**
>
> Putting your fish in `student-images/` keeps the curated library clean and makes it obvious which photos are yours vs. ours.

Step B1 — Name your file properly

Filename rules — these matter because R is sensitive to spaces and capital letters:

Good: `pomacanthus-imperator.jpg`, `chaetodon-auriga.jpg` **Bad:** `Pomacanthus Imperator.JPG` (capitals + space), `IMG_1234.jpeg` (uninformative), `cool fish.png` (space, .png will need extra step)

Format: lowercase, hyphenated, **`.jpg` extension**, named after the species (`genus-species.jpg`).

If you're not sure of the species name, look it up on FishBase or Wikipedia first. If you have multiple photos of the same species, append a number: `chaetodon-auriga-1.jpg`, `chaetodon-auriga-2.jpg`.

Step B2 — Get the file into `images/student-images/`

Docker users — the path of least resistance

Your lab repo on your Mac and the RStudio session inside the container are connected. **Anything you save into the host folder appears inside RStudio automatically. No "upload" needed.**
Step-by-step (Mac):

1. Open **Finder**.
2. From the top menu: **Go → Home** (or press `Cmd-Shift-H`). You see your home folder.
3. Double-click **`eeb187-pavo-lab`** → **`images`** → **`student-images`**.
4. **Drag your renamed JPG into that folder.**
5. Switch back to RStudio in your browser. In the Files pane (bottom-right), click the small refresh icon ( ). You should see the new JPG appear.
6. **Verify** in the R Console:

```
list.files("images/student-images")
```

   Your file should be in the list.

For Windows, replace step 2 with **Open File Explorer → `Documents → eeb187-pavo-lab → images → student-images`**.
For Linux, use your file manager to navigate to `~/eeb187-pavo-lab/images/student-images/`.

Binder users — use the Upload button

Binder is cloud-based, so you can't drag files into a Mac/Windows folder. Instead use RStudio's built-in upload:

1. In RStudio's **Files pane** (bottom-right), click the folder names to navigate: `images` → `student-images`. The breadcrumb at the top of the Files pane should now read `Home > images > student-images`.
2. In the same Files pane toolbar, click **Upload** (third button, has an upward arrow icon).
3. A dialog appears. Click **Choose File**, pick your JPG from your computer, then click **OK**.
4. The file uploads and appears in the file list.

For multiple files at once: zip them on your computer first (right-click on Mac → *Compress*; on Windows → *Send to* → *Compressed (zipped) folder*). Upload the .zip — RStudio auto-extracts it.

90-minute session limit: Binder sessions evaporate after 90 minutes of inactivity, **deleting any files you uploaded**. Finish your analysis in one sitting, or — better — use the Docker path (recommended) which has no timeout.

Codespaces users

Same as Docker — drag-drop the JPG into the VS Code file explorer at `images/student-images/`. Files persist between visits.

Native (local R + RStudio) users

You already have everything on your laptop. Drop your JPG into `~/eeb187-pavo-lab/images/student-images/` using Finder/File Explorer/your file manager — same as your everyday file workflow.

Step B3 — Confirm you can see your file from R

Whichever path you used, after the file is in place, run this in the R Console:

```
list.files("images/student-images")
```

You should see your filename in the output. If not:

- **Docker:** check the file is actually in `~/eeb187-pavo-lab/images/student-images/` on your Mac/PC (use Finder/File Explorer)
- **Binder:** check the file appears in the RStudio Files pane (you may need to click the refresh icon)
- **Both:** verify the folder name is spelled `student-images` (with an "s", with a hyphen)

Step B4 — Segment the photo (background removal)

If your photo still has water, reef, tank glass, or any non-fish background, you need to remove the background before pavo can analyze it. Three paths to do this — see Section 9 below for the full instructions.

---

3. Step 0 — Load packages and helpers

This is the **most-skipped step** that causes 80% of student errors. Run it first, every time.

> ⚠ ALL FOUR lines must run after every session start
>
> If you restart R (e.g., **Session → Restart R**, or Docker container restart, or you reload Binder), the entire global environment is wiped — every package, every variable, every helper. You must re-run Step 0 in full. Running just *one* of the four lines is the most common reason students get "could not find function" errors mid-pipeline:

Line you skipped	Error you'll get
`library(pavo)`	`Error: could not find function "getimg"` (also `classify`, `adjacent`)
`library(magick)`	`unable to load shared object …magick.so` (or some plotting errors)
`library(tidyverse)`	`Error: could not find function "bind_rows"` (or `pivot_longer`, `ggplot`)
`source("scripts/helpers.R")`	`Error: could not find function "simple_coldists"` (also `pick_white_bg`, `segment_fish`, `composite_to_white`)

In the Console (or by clicking through the script in the Source pane):

```
library(pavo)               # loads getimg(), classify(), adjacent()
library(magick)             # image I/O backend pavo uses
library(tidyverse)          # bind_rows, pivot_longer, ggplot
source("scripts/helpers.R") # segment_fish, simple_coldists, pick_white_bg, …
packageVersion("pavo")
```

What you should see:

```
Linking to ImageMagick 6.9.12.98 …
  Attaching core tidyverse packages …
[1] '2.9.0'
```

The four `library` / `source` lines above all need to run cleanly. The `source("scripts/helpers.R")` line is silent (no output unless something's wrong) — but it's loading 6 helper functions: `segment_fish`, `composite_to_white`, `flip_to_left_lateral`, `simple_coldists`, `pick_white_bg`, `pick_col`. **Skip it and Step 3 of the analysis will fail.**

> 💡 Pro tip
>
> At the top of `walkthrough.R` you'll see these four lines as block comments. Highlight all four lines in the Source pane (or the whole Step 0 block) and press **Cmd-Enter** (Mac) / **Ctrl-Enter** (Windows / Linux) to send them all to the Console at once.

⚠ If Step 0 didn't work

Error	Fix
`there is no package called 'pavo'`	The package didn't install. Run `install.packages("pavo")` and watch the output. If it errors, post in the channel.
`unable to load shared object … magick.so`	macOS native: `brew install imagemagick`, then re-run `install.packages("magick")`.
`cannot open file 'scripts/helpers.R'`	Your working directory isn't the lab repo. Run `getwd()` and fix per Section 1.
`[1] '2.8.0'` (or older)	Update pavo: `install.packages("pavo")`.

---

4. Step 1 — Load the three demo fish

The 3 demo fish are in `images/demo/segmented/`. In the Console:

```
sergeant_major <- getimg("images/demo/segmented/sergeant-major.jpg")
cleaner_wrasse <- getimg("images/demo/segmented/cleaner-wrasse.jpg")
mandarin_fish  <- getimg("images/demo/segmented/mandarin-fish.jpg")
```

`getimg()` reads each JPG into pavo's `rimg` object — a 3D array (height × width × {R, G, B}). **You should not see any output** — silent success means it worked.

Confirm by inspecting one:

```
class(sergeant_major)          # "rimg" "array"
dim(sergeant_major)            # height × width × 3
attr(sergeant_major, "imgname")
```

Plot all three side-by-side:

```
par(mfrow = c(1, 3))
plot(sergeant_major); title("sergeant major")
plot(cleaner_wrasse); title("cleaner wrasse")
plot(mandarin_fish);  title("mandarin fish")
par(mfrow = c(1, 1))
```

The plots appear in the **Plots tab** (bottom-right pane, below the Files tab).

Predict before you measure

Before you compute anything, write down which fish you think will have:

- the **highest A** (most vertical-edge dominance)
- the **lowest A** (most horizontal-edge dominance)
- A near 1 (no directional bias)

Hint: A = vertical edges ÷ horizontal edges. Bars run vertically (so they CREATE vertical edges where they meet the body color); stripes run horizontally.

If Step 1 didn't work

Error	Fix
`cannot open the connection`	Working directory wrong. `getwd()` should print the lab repo.
`Error: object 'getimg' not found`	You skipped `library(pavo)`. Back to Step 0.
Plot is empty / weird	The image file is corrupted. Re-clone the repo.

---

5. Step 2 — Classify each image into k color classes

`classify(img, kcols = k)` runs **k-means on every pixel's RGB triplet**. Each pixel gets snapped to one of k color centroids; the result is a "classified image" where the pattern is reduced to k discrete regions.

Choose k based on visible color count + 1 (the +1 is for the white background that segmentation added):

```
sm_class <- classify(sergeant_major, kcols = 4)   # yellow, black, pale belly, WHITE bg
cw_class <- classify(cleaner_wrasse, kcols = 4)   # dark stripe, mid blue, light, WHITE bg
mf_class <- classify(mandarin_fish,  kcols = 6)   # 5 fish colors + WHITE bg
```

You'll see `Image classification in progress...` while each runs (~5 seconds each).

Inspect the palettes:

```
attr(sm_class, "classRGB")    # 4 rows of (R, G, B) values, one per class
```

One row will have R G B 1 — that's the **white background** class. Note its row number (1, 2, 3, or 4) — we'll need it in Step 3.

Plot before / after for one fish:

```r
par(mfrow = c(2, 1))
plot(sergeant_major); title("Sergeant major - original")
plot(sm_class);       title("Sergeant major - classified, k = 4")
par(mfrow = c(1, 1))
```

You should see the original photo replaced by 4 flat-color regions. The black bars should still be visible as a discrete class; the white background should be a uniform region.

> **Why k matters**
>
> Try `classify(sergeant_major, kcols = 2)` and replot. The yellow body and black bars **collapse into one class** because k=2 forces a binary split (fish vs. background). This destroys the pattern. **Choosing k too low is the most common student error in Part B.** Rule of thumb: count the visibly-distinct colors on the fish, add 1 for the white background.

> **If Step 2 didn't work**
>
Error	Fix
`Error: 'kcols' missing`	Typo — it's `kcols` (with the 's'), not `kcol`.
Classified image looks washed-out	k is too small. Try k+1.
Two near-identical rows in `classRGB`	k is too high. Try k-1.

---

6. Step 3 — Adjacency analysis (the heart of the lab)

This step **drops a regular grid** over each classified image, **counts the transitions** between color classes between adjacent grid cells, and **computes the seven Endler 2012 pattern metrics**.

For each fish you need three things first:

1. The pairwise **color distances** (so adjacent() can compute boundary strength metrics)
2. The **white background class index** (so adjacent() can exclude it)
3. The actual `adjacent()` call

```r
# Sergeant major
sm_cd  <- simple_coldists(sm_class)
sm_bkg <- pick_white_bg(sm_class)
sm_adj <- adjacent(sm_class, xpts = 200, xscale = 5,
                   coldists = sm_cd, bkgID = sm_bkg, exclude = "background")

# Cleaner wrasse
cw_cd  <- simple_coldists(cw_class)
```

```
cw_bkg <- pick_white_bg(cw_class)
cw_adj <- adjacent(cw_class, xpts = 200, xscale = 5,
                   coldists = cw_cd, bkgID = cw_bkg, exclude = "background")

# Mandarin fish
mf_cd  <- simple_coldists(mf_class)
mf_bkg <- pick_white_bg(mf_class)
mf_adj <- adjacent(mf_class, xpts = 200, xscale = 5,
                   coldists = mf_cd, bkgID = mf_bkg, exclude = "background")
```

`xpts = 200` means a 200×200 sample grid. `xscale = 5` is the body length in cm — only matters for distance metrics.

Each `adjacent()` call returns a one-row data frame with ~30 columns of metrics. Stack them:

```
fish_adj <- bind_rows(
  sergeant_major = sm_adj,
  cleaner_wrasse = cw_adj,
  mandarin_fish  = mf_adj,
  .id = "fish"
)
```

⚠ If Step 3 didn't work

Error	Fix
`could not find function "simple_coldists"`	You skipped `source("scripts/helpers.R")` in Step 0. Run it now.
`m_dS` and `m_dL` come back as NA	`coldists` not passed. Re-check the `coldists = sm_cd` argument.
All metrics look weird, A way off	`bkgID` set to wrong class. Confirm `attr(sm_class, "classRGB")[sm_bkg, ]` is roughly (1, 1, 1).
Slow (~5–10 sec per fish)	Normal. xpts=200 is the speed/accuracy sweet spot.

---

7. Step 4 — Extract the headline metrics

pavo's `adjacent()` returns ~30 columns. We focus on seven from Endler (2012):

Metric	Meaning
k	number of color classes
m	transition density — how *busy* the pattern is per unit area
A	vertical edges ÷ horizontal edges — **the headline metric**
Jc	color heterogeneity — how *evenly* colors are used
Jt	transition heterogeneity — how diverse the boundaries are
m_dS	mean chromatic boundary strength — how saturated edges are
m_dL	mean luminance boundary strength — how bright/dark edges are

```
metrics <- tibble(
  fish   = c("sergeant_major", "cleaner_wrasse", "mandarin_fish"),
  family = c("Pomacentridae", "Labridae", "Callionymidae"),
  k    = pick_col(fish_adj, c("k", "kcols")),
  m    = pick_col(fish_adj, c("m", "m_r")),
  A    = pick_col(fish_adj, c("A", "Asp")),
  Jc   = pick_col(fish_adj, c("Jc")),
  Jt   = pick_col(fish_adj, c("Jt")),
  m_dS = pick_col(fish_adj, c("m_dS", "Cs_dS")),
  m_dL = pick_col(fish_adj, c("m_dL", "Cs_dL")),
)
print(metrics)
```

`pick_col()` accepts a vector of candidate names and returns the first one that exists — pavo's column names vary slightly between releases, this makes the code robust.

You should see something like:

```
# A tibble: 3 × 9
  fish           family            k     m     A    Jc    Jt   m_dS   m_dL
  <chr>          <chr>         <dbl> <dbl> <dbl> <dbl> <dbl>  <dbl>  <dbl>
1 sergeant_major Pomacentridae     4 0.42  1.54  0.71  …      …      …
2 cleaner_wrasse Labridae          4 0.31  0.60  0.62  …      …      …
3 mandarin_fish  Callionymidae     6 0.55  1.02  0.81  …      …      …
```

Did your prediction hold? A > 1 for the bars, A < 1 for the stripe, A 1 for the reticulate.

Record these values in Part A's metric table on the worksheet (`lab-05-pavo-intro.qmd`).

---

8. Step 5 — Visualize the comparison

```
metrics %>%
  pivot_longer(c(m, A, Jc, m_dL), names_to = "metric", values_to = "value") %>%
  ggplot(aes(x = fish, y = value, fill = family)) +
    geom_col() +
    facet_wrap(~metric, scales = "free_y") +
    scale_fill_manual(values = c("Pomacentridae" = "#5BC0EB",
                                 "Labridae"      = "#3FB8AF",
                                 "Callionymidae" = "#FFC946")) +
    labs(title = "Pattern metrics across three reef families",
         x = NULL, y = "metric value") +
    theme_minimal(base_size = 14) +
    theme(legend.position = "bottom",
          axis.text.x = element_text(angle = 30, hjust = 1))
```

Four panels, three fish each. The **A panel** is the punchline — bars highest, stripe lowest, reticulate in the middle.

This completes **Part A** of the worksheet.

---

9. Part B — Your three fish

Now you do the same on three species from your assigned lineage. Same pipeline; just swap your image paths in.

Step B.1 — Pick three species

Browse `images/student-fish/<your-lineage>/` (already on your laptop) and pick:

- One with **strong vertical bars** → predict A > 1
- One with a **horizontal element** (stripe, band) → predict A < 1
- One that's **spotted, reticulate, or solid** → predict A ≈ 1

Write your three picks in the worksheet's Part B Step 1.

Step B.2 — Segment your own fish

If your three picks are from the pre-loaded `student-fish/<lineage>/` library, **they're already cropped and segmented** — skip ahead to Step B.3 and load them directly with `getimg()`.

If you uploaded your own raw photo (with water/reef/aquarium glass in the background), segment it first.

How to write the path to your fish image

What is a 'path' and how do I write one?

A **path** is just R's way of finding your file. In this lab, every path is **relative to the lab repo root** — meaning R looks for files starting from your `eeb187-pavo-lab` folder. You don't need to type your full computer's directory; just the part inside the lab repo.
The folder structure for images:

```
eeb187-pavo-lab/
  images/
      demo/                          ← demo fish (Part A)
      student-fish/<lineage>/        ← pre-loaded starter species
          chaetodon-auriga.jpg
          chaetodon-lunulatus.jpg
          …
      student-images/                ← YOUR uploads
          your-fish.jpg              ← whatever you put here
```

To refer to a file, write the path with **forward slashes** (/), starting from `images/`:

Image you want	Path you write
A pre-loaded chaetodontid	`"images/student-fish/chaetodontidae/chaetodon-auriga.jpg"`
A fish you uploaded yourself	`"images/student-images/my-fish.jpg"`

RStudio can complete paths for you. In the Console, type the opening quote and the first letter — `"i` — then press the **Tab** key. RStudio shows a dropdown of files and folders. Keep tabbing as you type to navigate folders. This is the easiest way to avoid typos.

Path 1 — Local with `segment_fish()` (recommended if installed)

This is the rembg-based approach — same one used to make the demo fish. Run from the R Console:

```
# If your fish is at images/student-images/my-fish.jpg:
seg_path <- segment_fish("images/student-images/my-fish.jpg")
my_fish  <- getimg(seg_path)
```

The output goes back to `images/student-images/my-fish-segmented.jpg`. `seg_path` will be that string, ready to pass to `getimg()`.

Don't know what to type for the path?

Easy way 1 — let RStudio find it for you: In the Files pane (bottom-right), navigate to your file, click the file name (just once, don't open it), then click **More → Copy Path...** in the Files pane toolbar. RStudio puts the relative path on your clipboard. Paste it inside `segment_fish("...")`.
Easy way 2 — use list.files() to read the name:

```
list.files("images/student-images")    # returns a character vector of files
```

You can copy a filename from the printed output.
Easy way 3 — Tab completion: type `segment_fish("im` then press Tab. RStudio fills in `"images/`. Press Tab again to drop into folders.

Path 2 — Web tool (no install)

Drag-drop your fish JPG into one of:

- remove.bg — fastest, no signup
- photoroom.com — also free
- clipdrop.co/remove-background — Stability-AI tool

Each returns a **transparent PNG**. Save the PNG into your `images/student-images/` folder (same as Step B2 in Section 2). Then in R:

```
seg_path <- composite_to_white("images/student-images/my-fish.png")
my_fish  <- getimg(seg_path)
```

Path 3 — Manual crop (always works, no segmentation tools needed)

Open the JPG in Preview.app (macOS) or Photos (Windows). Drag a tight rectangle around the fish, then **Tools → Crop** (Preview) or **Edit → Crop** (Photos). Save. Move the cropped JPG into `images/student-images/`. Then in R, **load directly** — skip the segment helpers:

```
my_fish <- getimg("images/student-images/my-fish-cropped.jpg")
```

Manual crop leaves some background pixels in the corners — your A values will be less dramatic than the demo (closer to 1). The directional A claim still holds; just smaller effect sizes.

Step B.3 — Run the same pipeline

```
my_fish  <- getimg(seg_path)                # path from Step B.2
plot(my_fish)                                # confirm head on LEFT (left-lateral)
                                             # if not: flip_to_left_lateral(seg_path); my_fish

my_k     <- 4                                # = visible fish colors + 1 (for white bg)
my_class <- classify(my_fish, kcols = my_k)
plot(my_class)                               # does this match the original pattern?

my_cd    <- simple_coldists(my_class)
my_bkg   <- pick_white_bg(my_class)
my_adj   <- adjacent(my_class, xpts = 200, xscale = 5,
                     coldists = my_cd, bkgID = my_bkg, exclude = "background")

my_metrics <- tibble(
  fish = "your_species_name",
  k    = pick_col(my_adj, c("k", "kcols")),
  m    = pick_col(my_adj, c("m", "m_r")),
  A    = pick_col(my_adj, c("A", "Asp")),
  Jc   = pick_col(my_adj, c("Jc")),
  Jt   = pick_col(my_adj, c("Jt")),
  m_dS = pick_col(my_adj, c("m_dS", "Cs_dS")),
  m_dL = pick_col(my_adj, c("m_dL", "Cs_dL")),
)
print(my_metrics)
```

Do this **three times** (once per fish), record the values in the worksheet's Part B metric table.

> Manual-crop fallback uses different code
>
> If you used **Path 3 (manual crop)** in Step B.2, your image doesn't have a white background class. **Drop the `bkgID = my_bkg, exclude = "background"` arguments from `adjacent()`**:
>
> ```
> my_adj <- adjacent(my_class, xpts = 200, xscale = 5, coldists = my_cd)
> ```
>
> Your A values will be slightly less dramatic than the demo (closer to 1) because background pixels pollute the analysis. Note this in your worksheet — the rubric accommodates it.

---

10. Step C — Compare your fish to the demo

Look at your A values vs. the demo fish A values:

Demo fish	Family	Pattern	Demo A
Sergeant major	Pomacentridae	Vertical bars	~1.5
Cleaner wrasse	Labridae	Horizontal stripe	~0.6
Mandarin fish	Callionymidae	Reticulate	~1.0

For each of your three fish, ask:

1. Did the A statistic correctly classify it by pattern orientation? (A > 1 for your bar fish, A < 1 for your stripe fish, A 1 for your reticulate fish)
2. Which demo fish does each of yours most resemble in metric space?

Answer these in the worksheet's **Part B Step 4**.

---

11. Step D — Biological interpretation

Pick **one** of your three fish and propose a **selection pressure** that might have favored its pattern. The worksheet gives you a menu:

- crypsis (camouflage)
- aposematism (warning coloration)
- species recognition
- sexual signaling
- predator deflection (eyespots)
- disruptive coloration
- "uncertain — here's why"

The strongest answers **link your hypothesis back to specific measured metrics**. For example: "high m_dS suggests the saturation contrast at edges is large, which is consistent with aposematic warning rather than crypsis (which would predict low m_dS)."

Write 4–6 sentences in the worksheet's **Part B Step 5**.

---

12. Submit

Render or export the worksheet (`lab-05-pavo-intro.qmd`) with your tables filled in:

- **HTML/PDF** — Click **Render** at the top of RStudio's source pane
- **Or** copy the metric tables + your text answers into a Google Doc

Submit on BruinLearn by end of day.

Common gotchas (read this if you're stuck)

Symptom	Likely cause	Fix
`could not find function "getimg"` (or `classify`, `adjacent`)	You didn't `library(pavo)` after a session restart	Re-run Step 0 in full — all four lines
`could not find function "simple_coldists"` (or `pick_white_bg`, `pick_col`, `segment_fish`)	You didn't `source("scripts/helpers.R")` at session start	Re-run Step 0 in full — all four lines
`could not find function "bind_rows"` (or `pivot_longer`, `ggplot`)	You didn't `library(tidyverse)`	Re-run Step 0 in full — all four lines
`cannot open the connection` from `getimg()`	Working directory wrong	`getwd()` should print the lab repo path. Fix per Section 1.
Classified image is just two colors	k too small	Increase k by 1
Classified palette has duplicate-looking rows	k too large	Decrease k by 1
`A` for vertical-bar fish came back < 1	Image is right-lateral (head on right), or k chose the wrong palette	(a) `flip_to_left_lateral(seg_path)` and re-load; (b) try a different k
All metrics are NA after `adjacent()`	Image is empty (all-white or all-black)	Re-load with `getimg()` and check `plot(img)` actually shows the fish
`bkgID must be in 1:kcols`	`bkgID` from one fish was reused on another	Re-compute `pick_white_bg(...)` for each fish
Plots don't appear	Plot pane is closed or in a weird state	In RStudio: **View → Panes → Console on Right** to reset

---

What's next

There will be a **second pavo session** later in the term where we:

- Build a metric matrix across many species per lineage
- Cluster fish by pattern
- Compare clusters to phylogeny

That's the comparative-analysis follow-up. Today's lab gave you the tool; next time we use it for comparative biology.

Questions? Post in the EEB 187 channel. Most things you'll hit are documented above — search this guide first.